STACK-AND-DELAY: A NEW CODEBOOK PATTERN FOR MUSIC GENERATION

Gael Le Lan *Varun Nagaraja* *Ernie Chang*
David Kant *Zhaoheng Ni* *Yangyang Shi* *Forrest Iandola* *Vikas Chandra*

Meta AI

arXiv:2309.08804v1 [eess.AS] 15 Sep 2023

ABSTRACT

In language modeling based music generation, a generated waveform is represented by a sequence of hierarchical token stacks that can be decoded either in an auto-regressive manner or in parallel, depending on the codebook patterns. In particular, flattening the codebooks represents the highest quality decoding strategy, while being notoriously slow. To this end, we propose a novel stack-and-delay style of decoding strategy to improve upon the *flat* pattern decoding where generation speed is four times faster as opposed to vanilla flat decoding. This brings the inference time close to that of the *delay* decoding strategy, and allows for faster inference on GPU for small batch sizes. For the same inference efficiency budget as the *delay* pattern, we show that the proposed approach performs better in objective evaluations, almost closing the gap with the *flat* pattern in terms of quality. The results are corroborated by subjective evaluations which show that samples generated by the new model are slightly more often preferred to samples generated by the competing model given the same text prompts.

***Index Terms*—** music generation, audio generation, efficient decoding, transformer decoder

1. INTRODUCTION

The task of text-to-music generation has seen an increasing interest from the research community in the past year [1, 2, 3, 4, 5, 6]. This was enabled by the emergence of two competing architectures originating from the computer vision and natural language processing spaces, respectively: diffusion [7, 8] and Transformer-based language models (LMs) [9, 10]. The former method can be referred to as parallel decoding while the latter is usually auto-regressive.

The level of quality is getting closer to that of original songs, paving the road towards new commercial use cases such as personalized on-device music generation, where the batch size is typically small. However those models often come with a quality trade off: the higher the quality, the slower the generation and vice versa [3, 6]. During inference, the decoding strategy, hardware and model size influence the speed of the generation. [4] recently proposed a single-stage auto-regressive Transformer decoder that models sequences of compressed discrete music representations (i.e. tokens compute by an audio compression model [11]). The authors explored several codebook patterns for the discrete tokens sequence modeling. In particular, they showed that the best performing pattern relies on flattening the token stack (which will be referred to as the *flat* pattern in the rest of the paper). Indeed each piece of generated waveform is actually represented by not only one token but several, corresponding to the number C of residual projections in the Residual Vector Quantizer (RVQ) [12] module of the compression model.

Flattening the token stack comes with the cost of generating (and training) for a C times longer sequence, which implies a significantly higher real-time-factor (RTF), making the model unusable in practice for interactive user experience. To overcome that issue, the proposed *delay* pattern [4] was shown to be a good trade off between speed and quality.

In this paper we hypothesize that despite its efficiency, the *delay* pattern could affect the model ability to generate high quality samples by design. Starting from the stronger but slower *flat* pattern, we propose a new strategy called *stack-delay* that is able to generate music as fast as the original *delay* strategy, with significantly higher quality. The contributions of this paper are:

- a new *stack* codebook pattern that inherits the quality of *flat* while being faster and memory efficient during inference by reducing the past key/value streaming cache footprint.
- a new *stack-delay* pattern that:
  - benefits from the *stack* pattern strengths while being as fast as the *delay* pattern for generation.
  - produces higher quality music than *delay*, shown by objective and subjective evaluations.
- an new decoding schedule that involves interleaving decoded positions that prevents the model from decoding adjacent positions until they have enough context.

2. STACK-DELAY CODEBOOK PATTERN

2.1. Music generation

Given a text description, a sequence of text embeddings computed by the T5 encoder [13] serves as the conditioning signal

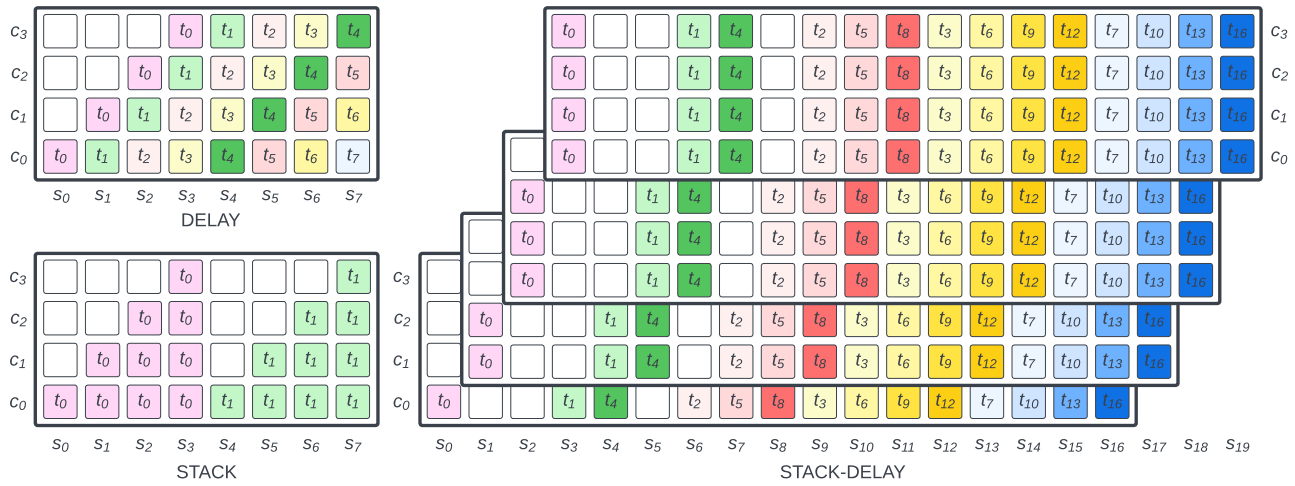

Fig. 1. Comparison of the proposed *stack-delay* pattern (right) with the *delay* (top left) and *stack* (bottom left). Under the *stack-delay* pattern the tokens are generated in a multi-stream fashion, in parallel. Time steps are decoded in a permuted manner. Only key/value embeddings from the top-level stream are stored in long-term streaming cache, which makes inference as efficient as *delay* while retaining better quality from *stack* pattern.

for a Transformer decoder model (using cross attention). The model generates a sequence of EnCodec [11] token stacks $\{c_{it}\}_{i=0}^{C-1}$ that are CNN-decoded into an audio waveform. i represents the token level while t represents the time step in the generated sequence.

In this paper we only consider the auto-regressive Transformer decoder architecture [9] that emits a probability distribution over the token space that is conditioned on the previously generated tokens (causal self attention in the Transformer decoder). During inference, the past self attention keys and values are stored in a streaming cache to optimize the generation time. Depending on the tokenizer framerate f (e.g. $f = 50Hz$), the duration of audio to generate d and the size of the token stack C (e.g. $C = 4$), the model has to generate $f \times C \times d$ tokens in a given amount of decoding steps that depend on the token codebook pattern and decoding schedule. The decoding schedule can be formalized as a function $\mathcal{G}(i, t)$ defining the decoding step for each c_{it}.

2.2. Codebook patterns

Contrary to the text domain, a segment of audio is not represented by a single token but by a stack of hierarchical tokens computed by quantizing [12] the latent embedding of a CNN auto-encoder [11]. This usually means the lower the token in the stack, the more information it carries. To address the issue of predicting tokens in a hierarchical manner, several codebook interleaving patterns have been explored [14, 4, 15], with the common idea to decode the lowest level token first then handle the higher levels in further decoding steps, which is the case for both auto-regressive (AR) [4] and non auto-regressive (NAR) [6] decoding architectures. Namely the decoding schedule is constrained such that:

$$\mathcal{G}(0, t) < \mathcal{G}(i, t), \forall i \in [1, C[\tag{1}$$

2.2.1. Delay

Regarding music generation, the *delay* interleaving pattern (presented on the top left part of Figure 1) was shown to be a good compromise between quality and AR decoding step count. Under the *delay* pattern, the C codebook levels are predicted in parallel but with a shift of in the decoded time step. Namely $\mathcal{G}(i, t) = t + i$. This means that each subsequent time step in the sequence starts to be decoded with only partial knowledge of the previous adjacent time step. For example, the prediction of c_{0t_1} in decoding step s_1 in the Figure is only conditioned on c_{0t_0}, previously decoded in s_0, but not on higher levels $\{c_i\}_{i=0}^{C-1}$ of time step t_0.

2.2.2. Stack

[4] showed that to obtain the highest music quality, flattening the codebooks performed the best, at the expense of C times more decoding steps.

$$\mathcal{G}(i, t) = C \times t + i < C \times T \tag{2}$$

This can be easily explained by the fact that subsequent decoded time steps benefit from the full context of the preceding ones. In such case the prediction of c_{0t+1} is effectively conditioned on $c_{[0,C-1][0,t]}$. The context length is C times bigger

pattern	decoding steps	context length
delay	T	T
flat	$T \times C$	$T \times C$
stack	$T \times C$	$T + C$
stack-delay	T	T

Table 1. Required decoding step count and maximum context length of the streaming cache during inference, as a function of the sequence length to generate $T = d \times f$ and the number of codebook levels C.

than *delay* since the at most $C \times T$ past Transformer self attention key/value representations are stored in the streaming cache during inference. To reduce the cache size we adapt the *flat* pattern by retaining and stacking the lower level tokens throughout the decoding process, as shown in Figure 1. Once a full stack has been decoded for a given time step, partial stacks can be erased from the streaming cache as the full stack contains all the required information. This way the maximum cache length is only of $C+T$ instead of $C \times T$. The *stack* pattern requires a customized attention mask during training that simulates the inference dynamic caching behavior. However it still requires C times more decoding steps than *delay*.

2.2.3. Stack-delay

To compensate for the increased decoding step count (i.e. inference time) of the *stack* pattern, we propose to introduce C parallel decoding streams in what we call the *stack-delay* pattern, illustrated in the right part of Figure 1. Having C parallel streams decoding a C times longer sequence means that overall the total number of decoding steps is the same as for the *delay* pattern (i.e. T). The main difference with *delay* is that we no longer stack tokens from different time steps but always from the same time step. This also allows positional encoding to encode not only the decoded time step but also the decoded token level, hence hinting the model about which time step and level is about to be decoded. We hope this will improve the overall model performance for the same inference efficiency budget as *delay*, due to the use of parallel-optimized compute hardware. We report the decoding step count and maximum context length in Table 1 for each pattern.

2.2.4. Timesteps interleaving

Finally, we introduce time steps permutation in the decoding schedule: the decoding remains auto-regressive but the model is trained to predict the token sequence in a time step-permuted order. This aims to offer more context for adjacent time steps decoding. An example of such interleaving pattern is shown on the right part of Figure 1, which corresponds to the decoding schedule defined in equation 3 with $k = 3$. According to the equation, the *delay* pattern decoding schedule corresponds to the case where $k = 1$.

$$\mathcal{G}(i, t) = t + (t \mod (k+1)) \times (k-1) + i \qquad (3)$$

3. EXPERIMENTAL SETUP

Most of the experimental setup follows that of MusicGen [4], we refer the readers to it for more details.

3.1. Model

The tokenizer is an EnCodec model [11], made of CNN autoencoder and Residual Vector Quantization module applied to the latent representation of waveforms. The RVQ module is made of $C = 4$ quantizers, each with a codebook size of 2048. It encodes 32 kHz monophonic audio into a stack of 4 tokens every 20ms (50 Hz framerate).

The Transformer decoder is made of 300M parameters, implemented with a customized version of *audiocraft*[1]. It uses Pytorch 2.0[2] flash attention for faster training and generation with optimized memory footprint. The model is trained on 30-seconds random crops of the full track. The models are trained for 200 epochs (400k steps) with the AdamW optimizer, a batch size of 192, β_1 = 0.9, β_2 = 0.95, a decoupled weight decay of 0.1 and no gradient clipping. A cosine learning rate schedule with a warmup of 4000 steps is used at the beginning of training. Models are trained with an exponential moving average with 0.99 decay. Training uses *fp16* mixed precision and distributed data parallelism on 24 A100 GPUs.

3.2. Generation

At each decoding step the Transformer decoder emits a probability distribution over the token space for time steps and levels to decode according to the decoding schedule. Tokens are sampled from the distribution with top-k nucleus sampling with $k = 250$ tokens and a temperature of 1.0. We apply classifier-free guidance [16] when sampling from the model's logits, with a guidance scale of 3.0.

The baseline model uses the *delay* codebook pattern from [4]. This translates 30 seconds of audio into $T = 500$ auto-regressive steps. For text conditioning, we use the T5 [13] text encoder. During training we drop the text condition with a probability of 0.1. We experiment with *flat*, *stack* and *stack-delay* codebook patterns.

3.3. Data

We train our models on 20K hours of licensed music: an internal dataset of 10K high-quality music tracks and the ShutterStock and Pond5 music data collections[3] with respectively 25K and 365K instrument-only recordings. All recordings are sampled at 32 kHz and come with a textual description. The models are evaluated on an in-domain split different from that of [4] and on the MusicCaps dataset [17].

[1] https://github.com/facebookresearch/audiocraft
[2] https://pytorch.org/
[3] www.shutterstock.com/music and www.pond5.com

pattern	in-domain			MusicCaps	RTF
	FAD	KLD	CLAP	FAD	(A100)
delay	0.69	0.48	0.36	4.91	1.07
flat	0.42	0.47	0.37	5.25	4.69
stack	**0.38**	0.48	0.37	5.16	4.77
stack-delay	0.48	0.48	0.37	**4.88**	1.13

Table 2. Quality/efficiency trade off of the proposed token sequence patterns for 30 seconds generated tracks.

decoding schedule $\mathcal{G}(i,t)$	FAD	KLD	CLAP
$t+i$ (*delay*)	0.45	0.50	0.38
$t+i$ (*stack-delay*)	0.43	0.51	0.37
$t+(t \mod 3)\times 1+i$	0.42	0.50	0.37
$t+(t \mod 4)\times 2+i$	0.36	0.51	0.38
$t+(t \mod 5)\times 3+i$	**0.34**	0.52	0.38

Table 3. Ablation study on the effect of permuting timesteps in the decoding schedule of the *stack-delay* pattern, for 10s samples on the in-domain evaluation dataset.

3.4. Evaluation

The different models are evaluated through a set of generated samples from a list of evaluation text-prompts. For objective evaluation we compute Frechet Audio Distance (FAD) using VGG classifier [18], Kullback–Leibler divergence (KLD) using PaSST model [19], and CLAP similarity score [20]. For subjective evaluation we run a blind pairwise comparison test where we present the evaluator two samples generated by two models but using the same text prompt, for a list of 20 text prompts. The human evaluators are asked to select the preferred sample from each pair based on perceived quality. Finally we report the RTF computed on A100 GPU when generating one sample (effective batch size of 2 from the model perspective due to classifier free guidance).

4. RESULTS

4.1. Baselines - *flat* and *delay* patterns

We consider two baselines: *flat*, which is known to produce the highest quality audio although requiring much more compute than *delay*, and *delay*, a good compromise between speed and performance, achieving a RTF close to 1, potentially unlocking streaming scenarios. *flat* achieves an in-domain FAD of 0.42, 39% lower than *delay*, while KLD and CLAP remain close. Despite its higher quality the RTF is above 4.

4.2. *Stack* pattern

We first investigate the *stack* pattern as a replacement for the (so far) state-of-the-art *flat*. Our results indicate that it is competitive with *flat*, even outperforming its FAD score with 0.38, with a similar RTF. The better FAD score indicates that the shorter required context length for generation might have a positive effect on music quality for long samples generations.

4.3. *Stack-delay* pattern

When considering the *stack-delay* pattern, our results indicate that it outperforms *delay* with a FAD of 0.48, although not as low at *stack*, but much more efficient with almost the same RTF as *delay*, unlocking potential real time streaming scenarios with better quality than the baseline. For subjective evaluation we only compare the *stack-delay* and *delay* versions. Our results indicate that samples generated by the *stack-delay* are preferred 51.3% of the time compared with *delay*. Such a small difference is to be expected given the small scale of our subjective evaluation.

4.4. Ablation - permuting decoded time steps

Finally, we look into the interleaved time steps decoding schedules defined in section 2.2.4. The ablation results are presented in Table 3 that compares four different schedules applied with the *stack-delay* pattern, and also including the *delay* baseline.

The table shows that the higher the decoding step count separating adjacent positions, the lower the FAD, with KLD and CLAP scores in a similar range. This shows the benefit of permuting the time steps in the *stack-delay* pattern. Without permutation (i.e. following the same ascending time steps schedule as *delay*), the *stack-delay* pattern only achieves marginal improvement. We also tried applying the *delay* pattern with the same permuted schedules and the performance was only on par with the baseline, which means that the combination of the proposed pattern and permuted decoding schedule is essential for better performance.

5. CONCLUSION

We introduce a new codebook pattern that relies on stacking the discrete music tokens, delaying/shifting the decoding of subsequent levels, and permuting the order of time steps to decode in the decoding schedule. The combination of the three outperforms the *delay* baseline quality-wise with a in-domain FAD reduction of 45% for the same inference efficiency budget, due to parallel decoding that compensates for an increased sequence length. We also show that stacking the tokens should be preferred to flattening them best when the highest quality is a priority. Finally the ablation study shows that time step permutation is key to achieve optimal performance, indicating that decoding of adjacent positions with only partial knowledge of previous time steps probably affects the performance of the *delay* pattern. Overall we hope our findings can help design better non-autoregressive decoding strategies in the future.

6. REFERENCES

[1] Flavio Schneider, Zhijing Jin, and Bernhard Schölkopf, "Mo\^usai: Text-to-music generation with long-context latent diffusion," *arXiv preprint arXiv:2301.11757*, 2023.

[2] Qingqing Huang, Daniel S Park, Tao Wang, Timo I Denk, Andy Ly, Nanxin Chen, Zhengdong Zhang, Zhishuai Zhang, Jiahui Yu, Christian Frank, et al., "Noise2music: Text-conditioned music generation with diffusion models," *arXiv preprint arXiv:2302.03917*, 2023.

[3] Max WY Lam, Qiao Tian, Tang Li, Zongyu Yin, Siyuan Feng, Ming Tu, Yuliang Ji, Rui Xia, Mingbo Ma, Xuchen Song, et al., "Efficient neural music generation," *arXiv preprint arXiv:2305.15719*, 2023.

[4] Jade Copet, Felix Kreuk, Itai Gat, Tal Remez, David Kant, Gabriel Synnaeve, Yossi Adi, and Alexandre Défossez, "Simple and controllable music generation," *arXiv preprint arXiv:2306.05284*, 2023.

[5] Peike Li, Boyu Chen, Yao Yao, Yikai Wang, Allen Wang, and Alex Wang, "Jen-1: Text-guided universal music generation with omnidirectional diffusion models," *arXiv preprint arXiv:2308.04729*, 2023.

[6] Hugo Flores Garcia, Prem Seetharaman, Rithesh Kumar, and Bryan Pardo, "Vampnet: Music generation via masked acoustic token modeling," *arXiv preprint arXiv:2307.04686*, 2023.

[7] Prafulla Dhariwal and Alexander Nichol, "Diffusion models beat gans on image synthesis," *Advances in neural information processing systems*, vol. 34, pp. 8780–8794, 2021.

[8] Huiwen Chang, Han Zhang, Lu Jiang, Ce Liu, and William T Freeman, "Maskgit: Masked generative image transformer," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 11315–11325.

[9] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al., "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, pp. 9, 2019.

[10] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al., "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[11] Alexandre Défossez, Jade Copet, Gabriel Synnaeve, and Yossi Adi, "High fidelity neural audio compression," *arXiv preprint arXiv:2210.13438*, 2022.

[12] Neil Zeghidour, Alejandro Luebs, Ahmed Omran, Jan Skoglund, and Marco Tagliasacchi, "Soundstream: An end-to-end neural audio codec," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 30, pp. 495–507, 2021.

[13] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *The Journal of Machine Learning Research*, vol. 21, no. 1, pp. 5485–5551, 2020.

[14] Chengyi Wang, Sanyuan Chen, Yu Wu, Ziqiang Zhang, Long Zhou, Shujie Liu, Zhuo Chen, Yanqing Liu, Huaming Wang, Jinyu Li, et al., "Neural codec language models are zero-shot text to speech synthesizers," *arXiv preprint arXiv:2301.02111*, 2023.

[15] Zalán Borsos, Matt Sharifi, Damien Vincent, Eugene Kharitonov, Neil Zeghidour, and Marco Tagliasacchi, "Soundstorm: Efficient parallel audio generation," *arXiv preprint arXiv:2305.09636*, 2023.

[16] Felix Kreuk, Gabriel Synnaeve, Adam Polyak, Uriel Singer, Alexandre Défossez, Jade Copet, Devi Parikh, Yaniv Taigman, and Yossi Adi, "Audiogen: Textually guided audio generation," in *The Eleventh International Conference on Learning Representations*, 2022.

[17] Andrea Agostinelli, Timo I Denk, Zalán Borsos, Jesse Engel, Mauro Verzetti, Antoine Caillon, Qingqing Huang, Aren Jansen, Adam Roberts, Marco Tagliasacchi, et al., "Musiclm: Generating music from text," *arXiv preprint arXiv:2301.11325*, 2023.

[18] Shawn Hershey, Sourish Chaudhuri, Daniel PW Ellis, Jort F Gemmeke, Aren Jansen, R Channing Moore, Manoj Plakal, Devin Platt, Rif A Saurous, Bryan Seybold, et al., "Cnn architectures for large-scale audio classification," in *2017 ieee international conference on acoustics, speech and signal processing (icassp)*. IEEE, 2017, pp. 131–135.

[19] Khaled Koutini, Jan Schlüter, Hamid Eghbal-Zadeh, and Gerhard Widmer, "Efficient training of audio transformers with patchout," *arXiv preprint arXiv:2110.05069*, 2021.

[20] Benjamin Elizalde, Soham Deshmukh, Mahmoud Al Ismail, and Huaming Wang, "Clap learning audio concepts from natural language supervision," in *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2023, pp. 1–5.